Bitacora de IA

Iteración	Objetivo	Prompt o enlace	Respuesta util	Error o supuesto	Decision del equipo
1	Comparar lost res patrones de lógica de negocio y analizar cual es el adecuado para nuestro caso.	Con base a los casos de uso, compara Transaction Script, Table Module y Domain Model. Declara tus supuestos y no elijas una opción hasta mostrar sus consecuencias.	-Transaction Script: rápido de escribir al inicio, pero CU-04 y CU-02 muestran el mayor riesgo — ambos dependen de invariantes que cruzan conceptos (fase + penalización; operador + disponibilidad), que Transaction Script no protege de forma centralizada. -Table Module: mejora la organización de datos, pero no resuelve el problema real del caso, que no es "¿cómo organizo mis tablas?" sino "¿cómo protejo reglas que dependen del estado y cruzan varios conceptos?". Sería la opción más débil de las tres para este caso específico. -Domain Model: mayor costo de diseño inicial (hay que definir bien los Aggregates antes de escribir código), pero es la única opción que resuelve de raíz los dos casos de uso de mayor riesgo (CU-02, CU-04), donde viven los invariantes más importantes del negocio.	Se asume que el volumen de servicios diarios es moderado (consistente con el supuesto ya documentado en el BRD: "monitoreo centralizado, no masivo"), por lo que el rendimiento bruto no es el criterio decisivo — lo es la mantenibilidad de las reglas. Se asume que el sistema seguirá creciendo (nuevas excepciones, nuevos tipos de cliente, nuevas reglas de tarifa), ya que la propia Unidad 2 mostró que las reglas se fueron descubriendo y corrigiendo de forma iterativa.	Elegir Domain Model como patrón de lógica de negocio para este caso.

2	Verificar si los elementos fueron correctamente identificados en el Mapa de Dominio (Entidades, Aggregates, Value Objects, etc.).	Las entidades serían: orden de servicio, cliente recurrente, operador. Value Objects: dirección, cotización, valor de la carga, ventana horaria. Aggregates: [Orden de servicio (raiz), orden de devolución, orden de cancelación, orden de cambio]. Domain Services: cotización, doble verificacion, penalización, caso especial. Domain events: orden de servicio confirmada, orden de cambio, orden de devolución, mercancia entregada, unidad averiada.	Orden de Servicio, Operador son correctas como identidades; ambas tienen identidad que se mantiene mientras cambian sus datos (folio, ID de operador). “Cliente recurrente” como entidad individual está mal planteado. La entidad es Cliente (con un atributo/tipo que distingue recurrente de ocasional), no “Cliente recurrente” aparte. Cotización NO es un Value Object ya que tiene identidad y ciclo de vida, lo que la convierte en una Entidad, y hay que borrarla de Domain Service. “Orden de cambio” y “Orden de devolución” están nombrados como el documento, no como el hecho. Corrígelos a lo que hacen.	La IA no considero tres Entidades más que participan dentro del mapa de dominio y de los casos de uso.	Se decidió corregir los errores marcados por la IA, además de agregar las Entidades de Unidad, Incidente/Inconformidad, Reclamación.
3	Detectar cuales metodos son necesarios y cuales podían ser sus posibles colaboradores	A partir de los casos de uso, cuales son tus propuestas de metodos y cuales considerarías serían sus respectivos colaboradores.	Ayudó a identificar los metodos según los casos de uso usando el formato “un método por caso de uso”, además de ayudar a identificar que necesidades delega cada método.	La IA utilizó nombres no correspondientes al mapa de dominio, tampoco agrego el Repositorio de Incidentes en el último método.	Se decidió corregir los nombres para que coincidan con el Mapa de Dominio, y se decidió agregar el Repositorio de Incidentes
4	Diseñar Repository, Unit of Work y CQRS para la Orden de Servicio (CU-02, CU-03 y CU-04).	Se compartió la división de trabajo y se pidió ayuda con la parte de Orden de Servicio: repositories, Unit of Work, Command y Query.	Tablas de repositories, transacciones por caso de uso, un Command y una Query con su DTO de lectura.	La Unidad 2 no define cuándo una unidad está disponible ni si "recolectado" cuenta como en ruta. Propuso un evento OrdenDeServicioCancelada que no estaba en la lista	Se registraron como supuestos pendientes de validar con el equipo.

				del mapa de dominio.	
5	Generar el esqueleto de código en Python.	Se indicó que el código base sería en Python ("usaremos python").	Estructura en dominio, aplicación e infraestructura, con pruebas que pasan.	Omitió que cancelar con unidad asignada requiere la autorización de Operaciones (US-008). Se detectó al cotejar el código con las User Stories.	Se agregó la autorización en el dominio, un error nuevo y 3 pruebas más (13 en total). El README y los comentarios del código se redactaron sin atribuir partes a personas.
6	Diseñar IncidenteRepository, el diagrama de clases, los diagramas de secuencia y el código base de AppSer_Incidente (CU-05).	Se compartió la división de trabajo y se pidió ayuda con la parte de Incidente: IncidenteRepository, diagrama de clases, 2 diagramas de secuencia y el esqueleto de AppSer_Incidente con su capa de dominio. Después se compartieron el BRD, el glosario, el BPMN y las User Stories de la Unidad 2 para cotejar el resultado, y se pidió pasar los diagramas a Miro.	Interfaz IncidenteRepository (guardar, obtener_por_id, obtener_abierto_por_orden), diagrama de clases separado por capas, secuencias de CU-05 (reasignación exitosa y cancelación a las 4 horas), código en Python con 5 pruebas que pasan y los diagramas en un tablero de Miro.	Al cotejar con la Unidad 2 se detectó que usó la fase "En ruta" y el glosario dice "En tránsito"; creaba el Incidente al reasignar, pero lo reporta el Operador (US-001); hacía que Operaciones disparara la cancelación, pero US-011 dice que el sistema cancela a las 4 horas; y dejaba la decisión del plazo en el Application Service. Además propuso un NotificadorCliente que no está en el informe y dio por regla "operador sin otra orden activa", que es una interpretación de N-07.	Se corrigieron las fases según el glosario, el incidente pasó a ser obligatorio, la cancelación se separó como método que dispara el sistema y la regla del plazo se movió al Domain Service Reasignación. Código, diagramas y Miro quedaron actualizados. Quedan pendientes el aviso al cliente con nuevos horarios (US-011) y confirmar que cancelar por falta de reemplazo no lleva penalización.
7	Redactar el ADR-002: Incidente como Aggregate independiente de Orden de Servicio.	Se pidió explicar qué es un ADR y redactar uno con base en la parte de Incidente, en una tabla en formato Word. Después se compartió la Unidad 2 para completar su contexto y su trazabilidad.	ADR-002 con los siete campos que pide la actividad, las alternativas (dentro del Aggregate de la orden, solo campos de avería o Aggregate propio) y el reparto de reglas entre Incidente, Orden de Servicio, Reasignación y AppSer_Incidente.	La primera versión dejaba abierto quién dispara el plazo de 4 horas y la Unidad 2 ya lo define (US-011: lo cancela el sistema). También citaba solo N-07 y N-09, sin US-001, US-011 ni el glosario.	Se actualizó el ADR con el contexto de US-001 y US-011 y la trazabilidad completa. Queda como Propuesta hasta que el equipo lo revise; lo pendiente es confirmar la penalización.